```
; Problem Statement: Write X86 ALP to find:
; a) Number of Blank spaces
; b) Number of lines
; c) Occurrence of a particular character.
; Accept the data from the text file. The text file has to be accessed during Program_1 execution and write FAR PROCEDURES in Program_2 for the rest of the processing.
; Use of PUBLIC and EXTERN directives is mandatory.

global far_proc          ; Declares the FAR procedure to be accessed externally

extern filehandle, char, buf, abuf_len   ; External references for the file handle, character, buffer, and buffer length

%include "macro2.asm"     ; Includes macros (e.g., for I/O operations)

;
*************************************************************
****
section .data
   nline      db 10,10                ; Newline characters (used for display purposes)
   nline_len    equ $ - nline              ; Length of newline data

   smsg       db 10,"No. of spaces are   : "  ; Message for spaces count
   smsg_len     equ $ - smsg

   nmsg       db 10,"No. of lines are    : "  ; Message for lines count
   nmsg_len     equ $ - nmsg

   cmsg       db 10,"No. of character occurrences are   : " ; Message for character occurrences count
   cmsg_len     equ $ - cmsg

;
*************************************************************
****
section .bss
   scount      resq 1    ; Space count (4-byte space to hold count of spaces)
   ncount      resq 1    ; Newline count (4-byte space to hold count of newlines)
   ccount      resq 1    ; Character count (4-byte space to hold count of occurrences of the character)

   dispbuff    resb 4    ; Buffer to hold the output in ASCII for displaying

;
*************************************************************
****
section .text
;   global _main
;_main:

far_proc:              ; FAR Procedure definition
   ; Initialize registers to 0
   mov   rax, 0       ; Clear rax (space counter)
   mov   rbx, 0       ; Clear rbx (newline counter)
   mov   rcx, 0       ; Clear rcx (character counter)
   mov   rsi, 0       ; Clear rsi (iterator for buffer)

   ; Load the character to search for (passed from Program 1) into bl
   mov   bl, [char]    ; Load the character to search for

   ; Load the buffer and buffer length (passed from Program 1) into registers
   mov   rsi, buf     ; Load buffer address into rsi
   mov   rcx, [abuf_len] ; Load actual buffer length into rcx

again:
   mov   al, [rsi]    ; Load the current byte (character) from buffer into al

case_s:
   cmp   al, 20h      ; Compare if the character is a space (ASCII 0x20)
   jne   case_n       ; If not space, jump to newline check
   inc   qword [scount] ; Increment space count
   jmp   next         ; Jump to next character

case_n:
   cmp   al, 0Ah      ; Compare if the character is a newline (ASCII 0x0A)
   jne   case_c       ; If not newline, jump to character check
   inc   qword [ncount] ; Increment newline count
   jmp   next         ; Jump to next character

case_c:
   cmp   al, bl       ; Compare if the character matches the search character
   jne   next         ; If not matching, jump to next character
   inc   qword [ccount] ; Increment character count

next:
   inc   rsi          ; Move to the next byte in the buffer
   dec   rcx          ; Decrement the counter (number of bytes to process)
   jnz   again        ; If there are still bytes left, repeat the loop

   ; Display results using the display16_proc
   display smsg, smsg_len    ; Display the message for spaces count
   mov   rbx, [scount]     ; Load the space count into rbx
   call   display16_proc   ; Call the display procedure for space count

   display nmsg, nmsg_len    ; Display the message for newline count
   mov   rbx, [ncount]     ; Load the newline count into rbx
   call   display16_proc   ; Call the display procedure for newline count

   display cmsg, cmsg_len    ; Display the message for character count
   mov   rbx, [ccount]     ; Load the character count into rbx
```

```
   call   display16_proc   ; Call the display procedure for
character count

   fclose [filehandle]     ; Close the file
   ret

;
**********************************************************
****
display16_proc:
   mov rdi, dispbuff       ; Point rdi to the display buffer
   mov rcx, 4              ; Set the loop count to 4 (for 4 digits to
display)
dispup1:
   rol bx, 4               ; Rotate bx left by 4 bits (for each
hexadecimal digit)
   mov dl, bl              ; Move lower byte of bx into dl
   and dl, 0fh             ; Mask upper digit of dl
   add dl, 30h             ; Add 30h to convert to ASCII
   cmp dl, 39h             ; If the ASCII value is greater than '9'
   jbe dispskip1           ; Skip adding 07h if less than or equal to
'9'
   add dl, 07h             ; Else, add 07h for proper ASCII
conversion

dispskip1:
   mov [rdi], dl           ; Store the digit in the buffer
   inc rdi                 ; Move to the next byte in the buffer
   loop dispup1            ; Decrement the counter and repeat if
not zero

   display dispbuff, 4     ; Display the buffer containing the
number (converted to ASCII)
   ret
```